



牛客竞赛
AC.NOWCODER.COM

牛客暑期多校

训练营

大豆





I - Bridge VI

简明题意：给定两两配对的选手及其初始得分，每场比赛可将 m 分任意分配给该组选手。求 Soy 在所有分配方案中可能取得的最好名次与最坏名次。

※之所以名字叫 Bridge VI，是因为该计分方法和桥牌的 VP 类似。当然，桥牌的 VP 并不能是任意实数，本题为了签到做了简化，有兴趣的读者可以尝试解答 VP 版的本题。

首个提交：待补

首个 **AC**：待补

通过率：待补

设 Soy 最终的得分为 x 。在 Soy 表现最好时，有 $x = a_1 + m$ ，最差时有 $x = a_1$ 。

考虑不和 Soy 一组的选手。设这一组选手的初始得分分别为 u, v 。首先考虑最多有多少人的得分超过 Soy。

若 $u + m > x, v + m > x, u + v + m > 2x$ ，则存在一组分配方式使得两选手得分都超过 Soy，贡献为 2。

若 $u + m > x$ 或 $v + m > x$ ，则存在一组分配方式使得存在一个选手得分超过 Soy，贡献为 1。

否则，贡献为 0。

接下来考虑最少有多少人得分超过 Soy。

若 $u > x$ 且 $v > x$ ，则两选手得分必然都超过 Soy，贡献为 2。

若 $u \leq x, v \leq x, u + v + m \leq 2x$ ，则不存在一组分配方式使得两选手得分超过 Soy，贡献为 0。

否则，贡献为 1。

然后考虑和 Soy 一组的选手。他的得分为 $a_2 + m - (x - a_1)$ 。与 x 比较即可得到贡献。

对每组选手分别计算以上的贡献即可，时间复杂度为 $\Theta(n)$ 。



G - Multiplication

简明题意：构造四个无前导零的十进制正整数，使 x_1, x_2 的前 a 位相同、 y_1, y_2 的前 b 位相同，而 x_1y_1 与 x_2y_2 的前 c 位不同，并满足长度限制。

※本题描述的 **Gallette** 只被召唤出头是对应原作的实际情况，有兴趣的读者可以玩一玩。

首个提交：待补

首个 **AC**：待补

通过率：待补



G - Multiplication (cont'd)

考虑让最终的乘积进位，即 $x_1y_1 = 999999\dots$ ， $x_2y_2 = 1000000\dots$

即可满足题设条件。

注意到 $\frac{10}{3} = 3.33333\dots$ ，因此我们可以作如下构造：

$$x_1 = x_2 = 300000\dots$$

$$y_1 = 333333\dots 3$$

$$y_2 = 333333\dots 4$$

只要不超过 $5 \cdot 10^5$ 位即可。



B - Deep Finesse

简明题意：统计 Ushio 在部分手牌已被指定时，从 1 到 $2n$ 中选择 n 张牌，使其无论 Fuuko 如何出牌都能逐轮用不大于对方的牌应对并获胜的方案数。

※本题的灵感和命名来源于桥牌的飞牌（**Finesse**），以及同名双明手分析软件，和某只大鲸鱼没有任何联系。

首个提交：待补

首个 **AC**：待补

通过率：待补



B - Deep Finesse (cont'd)

令：

$$a_i = \begin{cases} 1 & \text{Ushio 的手牌包含 } i, \\ -1 & \text{otherwise.} \end{cases}$$

则 Ushio 必胜的充要条件为：

$$\forall 1 \leq i \leq 2n : \sum_{j=1}^i a_j \geq 0$$

从直觉上说，这个式子的含义就是 Fuuko 每打出一张 -1 ，Ushio 都要从左侧寻找一个 1 来应对。



B - Deep Finesse (cont'd)

进行 DP。设 $dp[i][j]$ 为当前考虑 $a_1, a_2, \dots, a_i$ ，且前缀和为 j 时的

方案数，则：

$$dp[i][j] = dp[i-1][j-1] + \begin{cases} 0 & \text{Fuuko 指定了 } i, \\ dp[i-1][j+1] & \text{otherwise.} \end{cases}$$

边界 $dp[0][0] = 1$ ，最终的答案为 $dp[2n][0]$ ，时间复杂度为 $\Theta(n^2)$ 。



M - KV Cache

简明题意：给定一系列字符串请求与容量为 m 的 Trie 前缀缓存。处理请求会计算并缓存路径上的结点，超容量时只能删除叶结点；在已知全部未来请求时，求最小总计算代价。

※很明显，大模型服务商并不是通过 Trie 来缓存 KV Cache 的。

首个提交：待补

首个 **AC**：待补

通过率：待补



M - KV Cache (cont'd)

如果您上过或者看过操作系统这门课，应该会知道调页算法的 **Bélády OPT** 算法，也就是优先排出下次使用时间最晚的页。

对于本题来说，尽管存在了一个 **Trie** 的限制，但对于一对具有祖先关系的结点 $u \rightarrow v$ 来说，一定有 u 的下次使用时间不晚于 v ，因此更深的结点 v 会自动更优先排除。

因此，只需要通过优先队列维护当前 **Trie** 所有结点的下次使用时间即可，时间复杂度为 $\Theta(N \log N)$ ，其中 $N \leq 10^6$ 为 **Trie** 的结点数目。



H - It's a Magic, Not a Trick!

简明题意：给定非负整数数组。每次先任选一个元素加 1，再选择一个不小于 x 的元素减去 x ；求经过若干次合法操作后能够达到的最小数组元素和。

首个提交：待补

首个 **AC**：待补

通过率：待补



H - It's a Magic, Not a Trick! (cont'd)

若 $x = 1$ ，则 Himiko 无法释放任何魔法，答案为 $\sum a$ 。下面只讨论 $x \geq 2$ 的情况。

每轮操作都会使得 $\sum a$ 减小 $x - 1$ ，因此问题等价于求最多的操作次数。

对每个 a_i 做带余除法 $a_i = q_i x + r_i$ ，则无论 $+1$ 操作如何分配，总计 $p = \sum q_i$ 次减法操作永远都是可以成立的。在进行这些操作后，数组 a 就变为 $r_1, r_2, \dots, r_n$ ，以及可以自由分配的 p 次 $+1$ 。



H - It's a Magic, Not a Trick! (cont'd)

现在考察每个 $r_i > 0$ ，每个 r_i 需要 $c_i = x - 1 - r_i$ 次 +1 才能执行一次操作。为了使操作次数最大化，将 c_i 从小到大排序依次从 p 中购买。

购买完毕后，如果 $p > 0$ ，则还可以继续做 $\lfloor \frac{p}{x-1} \rfloor$ 次操作。得到操作总数 k 后，答案为 $\sum a - k(x - 1)$ 。时间复杂度为 $\Theta(n \log n)$ 。

Bonus: 解答 $+y$ 、 $-x$ 版本的本题。

K - AI Fine II

简明题意：两棵有根树的每个子树都有容量限制，每件饰品分别关联两棵树中的一个结点并带有权值。选择恰好 k 件饰品，同时满足两棵树的全部子树限制，并最大化权值和；若无解则输出 -1 。

※和 [CF2187F1 - AI Fine \(Maximizing Version\)](#) 一样，本题标题亦出自《交响乐之雨》的真结局路线。

首个提交：待补

首个 **AC**：待补

通过率：待补



K - AI Fine II (cont'd)

一个饰品会同时影响两棵树的对应结点，以及这些结点的所有祖先节点。因此它可以表示为：从源点出发，经过第一棵树、饰品边、第二棵树到达汇点的单位流。设源点为 S ，汇点为 T 。

对第一棵树 A ，连边 $S \rightarrow A_{root}$ ，容量为 $c_{A_{root}}$ 。对于父子结点 $u \rightarrow v$ ，连边 $u \rightarrow v$ ，容量为 c_v 。

对第二棵树 B ，连边 $B_{root} \rightarrow T$ ，容量为 $c_{B_{root}}$ 。对于父子结点 $u \rightarrow v$ ，连边 $v \rightarrow u$ ，容量为 c_v 。



K - AI Fine II (cont'd)

对一个饰品 (x, y, w) ，连边 $x \rightarrow y$ ，容量为 1，费用 w 。

对该图运行流量至多为 k 的最大费用最大流即可，若最大流小于 k ，则说明不存在方案，输出 -1 。否则，输出最大费用。时间复杂度为 $\Theta(kM \log M)$ ，其中 $M = \Theta(n + a + b)$ 为边数。

不要用 SPFA!!



E - Prefix Codes

简明题意：从字符串 s 的本质不同非空子串中选择一个非空集合；每个所选串都不能包含 p ，且任意两个所选串互不为前缀。求不同合法集合的数量。

※众所周知大豆不会 SAM，有 SAM 解法可以在题解区给出。

首个提交：待补

首个 **AC**：待补

通过率：待补

抛开 p 的限制不谈，有一个朴素的算法：将 s 的所有后缀插入进 **Trie** 中，则 **Trie** 的每个结点代表了一个本质不同子串。由于 **Trie** 的祖先关系代表了前缀关系，因此相当于求 **Trie** 的反链个数。在该 **Trie** 中进行树形 dp，对于结点 u ：

选择 u ，然后子树的所有结点都不选择。

不选择 u ，则方案数为各个子树之积。

即有：

$$dp[u] = 1 + \prod_{u \rightarrow v} dp[v]$$

现在考虑 p 的限制，定义 $lim[i]$ 为 $s[i..]$ 不包含 p 作为子串的最长前缀长度。这样，我们维护结点 v 对应的后缀起点 i ，就可以知道 v 是否包含 p 作为子串。若包含，则 $dp[v] = 0$ ，否则依然按照上式进行递推。

朴素的后缀 **Trie** 可以有 $O(n^2)$ 个结点，因此我们通过后缀数组可以建立**压缩后缀树**，那么递推式中各个简单 $dp[v]$ 的乘积也就变为各个压缩边的乘积。设 v 对应的后缀起点不包含 p 作为子串的最长前缀长度为 L ，深度为 $dep[v]$ 。



E - Prefix Codes (cont'd)

若 $L \geq dep[v]$ ，则结点 v 可以选择，那么方案包括选择压缩边 $u \rightarrow v$ 上 $dep[v] - dep[u] - 1$ 个隐式点之一，以及子树 v 的方案，即 $dp[v] + dep[v] - dep[u] - 1$ 。

若 $L < dep[v]$ ，则结点 v 不可选择，只能选择压缩边上前 $L - dep[u]$ 个隐式点之一，或者不选择，即 $\max\{L - dep[u], 0\} + 1$ 。

自底向上递推即可，最后答案为 $dp[root] - 1$ ，因为包含了空集。

时间复杂度为 $\Theta(\text{SA}(n) + m)$ 。



F - Tree Shifts Here

简明题意：给定一棵无权树。每次查询临时删除 u 到 v 的简单路径上的所有边，求所得各连通块直径之和；询问彼此独立。

※本题的标题 neta 自 Rewrite 的 BGM 《scene shifts here》

首个提交：待补

首个 **AC**：待补

通过率：待补



F - Tree Shifts Here (cont'd)

设每个查询路径为 $p_0, p_1, \dots, p_k$ ，则这些结点分别属于一个连通块，于是我们的目的就是求这些连通块直径之和。

首先考虑单个连通块的直径要如何表示。定义有向边 $x \rightarrow y$ 上的两个信息：

$F(x, y)$ 表示从 x 出发，沿着 $x \rightarrow y$ 方向能够到达的最远距离。

$G(x, y)$ 表示删去边后， y 一侧连通块的直径。

这两个信息都可以通过简单的树形 DP 求得。



F - Tree Shifts Here (cont'd)

于是，对于结点 x ，它所属连通块的直径为：

经过结点 x ，即两个最大的 $F(x, y)$ 之和。

不经过结点 x ，即最大的 $G(x, y)$ 。

两者的最大值。

由于每次最多删去两个邻居，因此对每个结点，需要维护前 4 大的 F 和前 3 大的 G 。



F - Tree Shifts Here (cont'd)

现在考虑一段路径，它需要维护路径上所有内部点连通块的直径之和。对于两个路径 A 和 B ，只需要将 A 的最后一个点和 B 的第一个点处理成新的内部点信息，就可以实现两路径的合并。多段路径的合并符合结合律。

由于该信息符合结合律，那就好办了。树上倍增，树剖等方式都可以将一段路径拆分为 $\Theta(\log n)$ 段，而每段路径的合并是 $\Theta(1)$ 的，因此总的复杂度就是 $\Theta(n \log n)$ 。



C - Fraction on a Ring

简明题意：给定质数 P 与上界 n 。对每个 $1 \leq a < b \leq n$ ，令 $x = ab^{-1} \bmod P$ ；统计满足 $\frac{a}{b} > \frac{x}{P}$ 的数对 (a, b) 。

首个提交：待补

首个 **AC**：待补

通过率：待补



C - Fraction on a Ring (cont'd)

固定分母 b ，令 $x = ab^{-1} \bmod P$ ，写成带余除法即 $bx = qP + a$ ，从

而有 $a \equiv -qP \pmod{b}$ 。由于 $\gcd(b, P) = 1$ ，变换

$f(q) = -qP \bmod b$ 在剩余环 $\mathbb{Z}_b$ 内是双射，因此我们可以枚举 q 。

将带余除法式代入原式整理后可以得到 $a > q$ ，从而问题转化为求

满足 $f(q) > q$ 的 q 个数。



C - Fraction on a Ring (cont'd)

若 $f(q) > q$ ，则有 $f(b - q) = b - f(q) < b - q$ 。因此对于 $f(q) \neq q$ 的 q 而言，恰有一半 q 满足 $f(q) > q$ 。问题进一步转化为求 $f(q) = q$ 的个数，即 $(P + 1)q \equiv 0 \pmod{b}$ 。

这是一个简单的一次同余方程，我们知道解集共有 $\gcd(b, P + 1)$ 个元素。从而，我们枚举 b ，满足 $f(q) > q$ 的 q 共有

$$\frac{1}{2} \sum_{b=1}^n (b - \gcd(b, P + 1))$$

个。

利用经典的欧拉反演，我们有

$$\sum_{b=1}^n \gcd(b, P+1) = \sum_{d|P+1} \varphi(d) \left\lfloor \frac{n}{d} \right\rfloor$$

从而答案为：

$$\frac{n(n+1)}{4} - \frac{1}{2} \sum_{d|P+1} \varphi(d) \left\lfloor \frac{n}{d} \right\rfloor$$

接下来用 **Pollard-Rho** 求解 $P+1$ 的所有因数，再计算欧拉函数即可。单组时间复杂度为 $O(P^{1/4} + \tau_0(P+1))$ ，其中 τ_0 表示 $P+1$ 的因子个数，最大约为 10^5 。



L - Shattered Minimum

简明题意：排列的若干区间构成独立组合游戏。每步击碎当前区间最小值，并选择只保留左段、只保留右段或同时保留两段；双方最优行动，判断先手是否必胜。

首个提交：待补

首个 **AC**：待补

通过率：待补



L - Shattered Minimum (cont'd)

对于一段区间，它的后继有三个：左段，右段，以及同时保留两段。设左段和右段的 **SG** 值分别为 x 和 y ，那么该段的 **SG** 值即为 $F(x, y) = \text{mex}\{x, y, x \oplus y\}$ 。由于后继数量不超过 3，因此任意一个状态的 **SG** 值仅取得 0, 1, 2 三个值。

对排列建立笛卡尔树，设 z 为区间 $[l, r]$ 最小值所在位置，它也是 $[l, r]$ “导出子图”的根。



L - Shattered Minimum (cont'd)

由于左右半的变换完全对称，我们仅讨论左半区间 $[l, z - 1]$ 的变换过程。

初始状态 $u := l$ 包含自己和它的完整右子树。由于空段的 **SG** 值是 **0**，所以初始状态的 **SG** 值为 $x = F(0, \text{SG}(\text{right}(l)))$ 。接下来开始向父节点 v 扩展区间。

若 u 是 v 的左儿子，那么 v 的右子树会被完整地扩展到，执行 $x := F(x, \text{SG}(\text{right}(v)))$ 。

若 u 是 v 的右儿子，此时 v 的左子树在范围外，无事发生。

如此上升，一直到 $\text{left}(z)$ 为止，就求得了 $[l, z - 1]$ 的 **SG** 值。



L - Shattered Minimum (cont'd)

最后再在 z 将左右子树的 **SG** 值合并，就得到了 $[l, r]$ 的 **SG** 值，
最终再通过异或合并所有 **SG** 值，得到哪一方先手必胜。

在一次上升中，**SG** 值会执行一次 $\{0, 1, 2\} \rightarrow \{0, 1, 2\}$ 的变换。我们可以分别维护前缀变换和后缀变换的倍增表，使得至多只需要上升 $O(\log n)$ 次。因此，时间复杂度为 $O((n + m) \log n)$ 。



J - Oni's Ring

简明题意：给定每个长度为 k 的二进制串在环形二进制串中的出现次数 c_x 。统计满足全部频数约束、且旋转视为相同的本质不同圆环数量。

首个提交：待补

首个 **AC**：待补

通过率：待补



J - Oni's Ring (cont'd)

建立 **de Bruijn 图** G ，每个二进制串 $b_1b_2 \dots b_k$ 对应有序边

$b_1b_2 \dots b_{k-1} \rightarrow b_2b_3 \dots b_k$ ，边重数为对应的 c_x 。这样，沿着图的某

个起点开始行走，每走过一条边就代表该边对应的二进制串出现

了一次，而一条欧拉回路就代表着一个满足所有边约束的圆环。

同时，这也表明圆环的长度 $N = \sum c$ 。

将每条重边独立编号，可以依 **BEST 定理** 求出所有欧拉回路的数目，再将重复的重边去重，可以得到带起点编号的二进制串数目：

$$Nt^{\text{root}}(G, u) \frac{\prod_{v \in V} (\deg(v) - 1)!}{\prod_x c_x!}$$

其中 u 为任意节点。

接下来处理旋转，令 $g = \gcd(c)$ ，则 **Burnside** 引理给出

$\frac{1}{N} \sum_{d|g} \varphi(d) F(d)$ ，其中 $F(d)$ 表示所有 c_i 均除以 d 后的带编号计数。



J - Oni's Ring (cont'd)

在出度矩阵 $L^{\text{out}}(G)$ 中，所有重边数目均除以 d ，等价于所有元素都除以 d 。令结点数 $s = 2^{k-1}$ ，于是 $t^{\text{root}}(G, u)$ 变为原本的 $1/d^{s-1}$ 。

故答案为：

$$\sum_{d|g} \varphi(d) \frac{t^{\text{root}}(G, u)}{d^s} \frac{\prod_v (\deg(v)/d - 1)!}{\prod_x (c_x/d)!}$$

求行列式的时间复杂度为 $O(8^{k-1})$ ，处理旋转的时间复杂度为 $O(2^k \tau_0(g))$ 。



A - 14 Minesweeper Variants

简明题意：给定至多 6×6 的扫雷真实棋盘，但每次点击必须仅凭当前公开线索证明该格在所有相容雷区中都安全。构造打开全部安全格的合法点击顺序，或判断无解。

首个提交：待补

首个 **AC**：待补

通过率：待补



A - 14 Minesweeper Variants (cont'd)

为了方便讨论，我们设棋盘第 0 行和第 $n+1$ 行为两行空白虚拟行。

一个数字只和上下三行的雷区排布有关。为了避免复杂的分类讨论，我们首先预处理一个集合 $T[c][d][p][q]$ ，它表示若上一行的雷区分布为 p ，中间一行的雷区分布为 q ，且中间行第 c 列数字为 d 时，下一行的 2^m 种雷区排列有哪些是可行的。由于 $2^m \leq 64$ ，我们可以将 T 压缩为一个 64 位整数。



A - 14 Minesweeper Variants (cont'd)

维护 $valid[i][p][q]$ ，表示当前可见的第 i 行线索能够使第 $i + 1$ 行有哪些雷区排列能够满足第 $i - 1$ 行 p 和第 i 行 q 。一旦有数字线索揭露，就可以根据 T 排除不合法的雷区分布。

维护 $allowed[i]$ ，表示第 i 行的哪些雷区排列保证了雷不出现在第 i 行已经打开的格子中。

A - 14 Minesweeper Variants (cont'd)

接下来进行前后缀分解：

定义前向 **dp** 数组 $pre[i][p][q]$ ，它表示在第 $i - 1$ 行和第 i 行的雷区分布为 p 和 q 时，行 $0, 1, \dots, i$ 可能的雷数之和，边界为 $pre[0][0][q] = 1 \ll \text{POPCNT}(q)$ 。

若第 $i + 1$ 行雷区分布 r 满足 $valid[i][p][q]$ 以及 $allowed[i + 1]$ ，则作转移：

$$pre[i + 1][q][r] \mid = pre[i][p][q] \ll \text{POPCNT}(r)$$

和背包非常类似。



A - 14 Minesweeper Variants (cont'd)

类似地，定义后向 dp 数组 $suf[i][p][q]$ 表示第 $i - 1$ 行和第 i 行的雷区分布为 p 和 q 时，行 $i + 1, i + 2, \dots, n$ 可能的雷数之和。

第 i 行的雷区分布 q 合法，当且仅当存在 x, y, p 使得：

$$x \in pre[i][p][q], \quad y \in suf[i][p][q], \quad x + y = M$$

其中 M 是雷数总和。从而我们可以记录到第 i 行有哪些 q 是合法的，将这些 q 中的坐标从安全格中删去。



A - 14 Minesweeper Variants (cont'd)

最后检查完所有的 (i, q) 之后，剩下的安全格就是绝对安全的。

预处理 T 的时间复杂度为 $O(m8^m)$ ，每次前后向 dp 的时间复杂度

是 $O(n8^m)$ ，至多需要 $O(nm)$ 次 dp，总的时间复杂度是

$O(n^2m8^m)$ 。这个上界非常宽松，实际运行远远达不到这么多。



D - Sea, you & coprime II

简明题意：给定正整数数组。若子数组中能选出四个不同下标，组成两对互质元素，则称其合法。每次查询 $[L, R]$ ，统计完全位于该区间内的合法子数组数量。

首个提交：待补

首个 **AC**：待补

通过率：待补



D - Sea, you & coprime II (cont'd)

称存在满足题意的四个下标的区间 $[l, r]$ 为好区间，反之则为坏区间。好、坏区间均具有单调性：

若 $[l, r]$ 是好区间，则 $[l - 1, r]$ 和 $[l, r + 1]$ 均为好区间。

若 $[l, r]$ 是坏区间，则 $[l + 1, r]$ 和 $[l, r - 1]$ 均为坏区间。

这允许我们使用滑动窗口对每个左端点 l ，求解最远的坏区间右端点。

将互质视为下标之间的连边，则一个区间是坏的，并且它不是空图，当且仅当它的所有非孤立点构成一个连通分量，并且是以下两种之一：

星形图（特别地，只有一条边也算星形图）；

三角形。

这两者均可以用度数唯一确定：

对于星形图，度数应为 $d, \underbrace{1, 1, 1, \dots, 1}_{d \text{ 个 } 1}, 0, 0, \dots, 0$ ；

对于三角形，度数应为 $2, 2, 2, 0, 0, \dots, 0$ 。

这使得仅通过维护度数判断是否是坏区间成为可能。

于是，我们需要动态求解下式：

$$\text{deg}(j) = \sum_{i=l}^r [\text{gcd}(a_i, a_j) = 1]$$

其中 j 是待加入或删除滑动窗口的下标。对每个整数 z ，我们动态维护当前滑动窗口的 $C_z = \sum_{i=l}^r [z \mid a_i]$ 。这样，设 a_j 的质因子集合为 P_j ，由容斥原理给出

$$\text{deg}(j) = \sum_{T \subseteq P_j} (-1)^{|T|} C_{\prod T}$$

这样就能在 $O(2^{\omega(a_j)})$ 时间内求出新的 $\deg(j)$ 。对于 $\deg(j) = 1$ ，我们还需要求出它的唯一邻居，它可以由 $\sum_{i=l}^r i[\gcd(a_i, a_j) = 1]$ 用类似的方法算出。

我们维护滑动窗口的四个状态：空图、单边、星形图、三角形，插入和删除操作均会使得滑动窗口在不同状态间转移。设 $[i \sim j] = [\gcd(a_i, a_j) = 1]$ ，即表示 i 和 j 是否有连边。

记 $d = \deg(j)$ ，插入新位置 j 的条件如下。

当前状态	空图	单边 $u - v$	以 c 为中心的星形图	三角形
仍为坏区间的充要条件	恒成立	$d = [j \sim u] + [j \sim v]$	$d = [j \sim c]$	$d = 0$
插入后的状态	$d = 0$: 不变; $d = 1$: 单边; $d \geq 2$: 以 j 为中心的星形图	$d = 0$: 不变; $d = 1$: 星形图; $d = 2$: 三角形	$d = 0$: 不变; $d = 1$: 将 j 加入叶子	不变

若插入失败，则将当前滑动窗口右端点记为当前 l 的坏区间最右端点，然后删除 a_l 后继续尝试插入。

删除位置 j 时，新区间仍然为坏区间，只需要进行如下的状态转移。

当前状态	空图	单边	星形图	三角形
删除后的状态	不变	$d = 0$: 不变; $d = 1$: 空图	$d \geq 2$: 空图; $d = 1$: 删去叶子, 若此时边数变为 1 则降级为单边; $d = 0$: 不变	$d = 2$: 单边; $d = 0$: 不变



D - Sea, you & coprime II (cont'd)

如是，我们得到了每个左端点 l 的极大坏区间右端点 $f(l)$ 。现在我们考虑一个查询 $[L, R]$ ，对于任意一个左端点 l ，右端点至少需要 $> f(l)$ 才能成为好区间，于是查询的答案为：

$$\sum_{l=L}^R \max\{R - f(l), 0\}$$

由于 $f(l)$ 单调不降，可以在线性时间内预处理每个 R 对应的 $f(l) \geq R$ 分界点及 f 的前缀和，从而在 $O(1)$ 时间内回答每个查询。



D - Sea, you & coprime II (cont'd)

记 $A = \max_i a_i$ 、 $\omega_{\max} = \max_i \omega(a_i)$ 、 $\tau_{\max} = \max_i \tau_0(a_i)$ 。预处理质因子需要 $O(A \log \log A)$ 时间。滑动窗口共变动 $O(n)$ 次；处理 a_j 时，计算度数与更新 C_z 分别需要 $O(2^{\omega(a_j)})$ 、 $O(\tau_0(a_j))$ 时间。本题中 $\omega_{\max} \leq 7$ 、 $\tau_{\max} \leq 240$ ，因此总时间复杂度为

$$O(A \log \log A + n (2^{\omega_{\max}} + \tau_{\max}) + q) .$$

THANKS

AC.NOWCODER.COM